# Book III — Cartridge and Expansion Reference

BMSX

# Table of contents

**About these manuals** **1**

**1 Cartridge sockets and mapped card devices** **3**

1.1 Card composition . . . 3
1.2 Boot selection . . . 3
1.3 Selection and execution . . . 3
1.4 Programming route . . . 4
1.5 Cartridge controller . . . 4
1.6 Card storage and MMIO access . . . 5
1.7 Cartridge mailbox . . . 5
1.8 Interrupts: two latches, two acknowledgements . . . 7
1.9 DMA between sockets . . . 7
1.10 Access faults . . . 9
1.11 Two-card programming example . . . 9

# About these manuals

The BMSX Developer Reference describes the console, its programming architecture and the interfaces available to cartridge software. The three books separate installed hardware, processor and firmware programming, and cartridge devices.

**Book I — System and Hardware Reference** introduces the machine and its memory system, drawing and display units, audio, direct memory access, interrupts and peripheral control. Start with the system overview, then consult the individual unit chapters for programming sequences and reference details.

**Book II — BLua32 and BIOS Reference** describes the processor, instruction set, BLua language extensions and resident BIOS services. It supplies the program and data representations needed to implement cartridge software.

**Book III — Cartridge and Expansion Reference** describes cartridge sockets and mapped card devices, including their memory and interrupt interfaces.

Addresses are byte addresses unless stated otherwise. Register descriptions give bit positions within the named word; binary layouts give byte offsets within the described structure. Capacities, units and synchronization conditions are specified with the interface to which they apply.

# 1 Cartridge sockets and mapped card devices

The BMSX cartridge interface connects two independently occupied sockets, numbered 0 and 1, to a shared set of CPU address windows. A card can supply program and resource ROM, writable RAM, a word mailbox, or a combination of these resources. A RAM- or mailbox-only card can extend a program running from the other socket.

Ordinary CPU data accesses use the selected socket. Instruction fetches retain their own execution-socket selection, and DMA can select a socket separately for each endpoint. A cartridge program can therefore access a second card without relocating its own image or redirecting its next instruction fetch. The selection and transfer sections below explain when each selection is taken.

## 1.1 Card composition

Cards can combine ROM, RAM and a mailbox independently. ROM supplies read-only program and resource data, RAM supplies writable space, and the mailbox provides data and control ports. Consult the card specification for its installed resources and capacities. STATUS reports which sockets are occupied.

| Resource | Use | Capacity |
|---|---|---|
| Socket | Connection for one card | Two sockets, numbered 0 and 1 |
| Card ROM | Program and read-only data | Card-specific, within the 512-MiB ROM window |
| Card RAM | Writable data | Card-specific, within the 15-MiB RAM window |
| Mailbox | Word exchange, interrupt and DMA-request control | DATA and CONTROL words with a local interrupt latch |

The address windows accommodate cards of different capacities. Addresses beyond an installed ROM or RAM return zero; the access section below describes tail reads and writes. Socket occupancy alone does not identify a bootable card.

## 1.2 Boot selection

At startup, the BIOS examines socket 0, then socket 1, selecting each present card in turn. It starts the first card with a usable executable ROM image. A RAM-only card or a card whose ROM does not meet the startup requirements is skipped.

Book I, “Memory and program storage”, under “ROM header fields” specifies the ROM header, startup requirements and entry sequence.

## 1.3 Selection and execution

The CPU retains an execution-socket latch for cartridge instruction fetches. SELECT controls ordinary data accesses independently, allowing a program to read the other card while continuing to execute from its original socket.

A cartridge-targeted `CP0.EXEC` samples the selected socket and replaces the instruction-fetch latch. Both sockets use the same cartridge addresses, so an image does not require relocation merely because it is inserted in the other socket. A closure retains a function-record address, not a socket: entering it uses the current execution-socket latch. See Book I, “Memory and program storage”, under “EXEC and cartridge entry” and Book II, “CPU and BLua32 instruction reference”, under “CP0 and execution control”.

## 1.4 Programming route

1. Establish the inserted card's composition and preserve any storage still in use. STATUS reports occupancy, not RAM capacity or mailbox presence.
2. Write SELECT for ordinary CPU accesses. Use aligned word stores for mailbox commands; selecting a socket neither resets nor enters it.
3. For mailbox interrupts, clear the local latch to rearm its next rising edge, and acknowledge the owned pending bit at the central controller separately.
4. For DMA, set both request levels, addresses, count and control before START. Use explicit socket request selectors when a pending transfer must remain independent of later CPU selection changes.
5. Wait for the channel's terminal state and inspect faults before using its output. Transport completion is not an application response from the card.

The following reference defines the retained words, write effects, request levels and access restrictions used by that sequence.

## 1.5 Cartridge controller

The controller's SELECT register chooses the socket used by ordinary data accesses. STATUS reports both presence bits and the selected socket, including when the selected socket is empty.

### 1.5.1 Controller registers

| Address | Name | Access | Controller-reset value |
|---|---|---|---|
| `0x08010420` | `SELECT` | Read/write | `0x00000000` |
| `0x08010424` | `STATUS` | Read-only | Installed-cartridge presence bits; bit 16 clear |

`SELECT` retains the complete raw 32-bit selection word. A read returns that word, including bits 31:1. Only bit 0 participates in ordinary socket selection. The other bits remain stored and readable; they neither select additional sockets nor report cartridge properties.

| Register | Bits | Meaning |
|---|---|---|
| `SELECT` | 0 | Ordinary socket selection: 0 selects socket 0; 1 selects socket 1. |
| `SELECT` | 31:1 | Retained raw bits; no effect on socket selection. |
| `STATUS` | 0 | 1 when socket 0 contains a cartridge. |
| `STATUS` | 1 | 1 when socket 1 contains a cartridge. |
| `STATUS` | 16 | Ordinary selected socket: 0 for socket 0; 1 for socket 1. |
| `STATUS` | 15:2, 31:17 | Read as zero. |

### 1.5.2 Selection and presence

A `SELECT` write takes effect for subsequent ordinary cartridge accesses. Both cards retain their device state, interrupt latches and DMA-request levels across the selection change.

An individual bus transaction may explicitly select a socket instead of using `SELECT`. Such an override does not modify the retained selection word. `STATUS` continues to report ordinary selection, not the socket chosen by that transaction.

The presence bits are independent of selection. Selecting an empty socket changes STATUS bit 16 while leaving bits 1:0 unchanged. Reads from that socket return zero and writes have no effect.

### 1.5.3 Controller reset

Controller reset clears the entire `SELECT` word, resets each present cartridge, and republishes DMA request lines for both sockets. Empty sockets contribute no asserted request lines. Reset does not change which sockets contain cartridges, so `STATUS` need not read as zero afterward. The controller itself neither raises nor clears an interrupt as part of this reset sequence; cartridge-specific reset behavior is defined separately.

## 1.6 Card storage and MMIO access

Each selected card uses the following address windows. ROM and RAM offsets are byte offsets from their respective bases; the card's installed capacity may be smaller than the window.

| Region | First address | Exclusive limit | Window size |
|---|---|---|---|
| ROM | `0x10000000` | `0x30000000` | 512 MiB |
| RAM | `0x30000000` | `0x30f00000` | 15 MiB |
| MMIO | `0x30f00000` | `0x31000000` | 1 MiB |

Absent or unpopulated ROM and RAM locations read as zero. Smaller memories are not mirrored through the remaining window. ROM writes are ignored; a RAM write takes effect only when its complete width fits the installed memory.

ROM and RAM scalar values are little-endian: the lowest-addressed byte occupies bits 7:0. For scalar ROM and RAM reads, the starting address selects the region. A multi-byte read remains a read of that selected region, including when fewer than the requested number of installed bytes remain. Available bytes occupy the low-order portion of the result; missing high-order bytes are zero. This differs from a byte sequence spanning address windows, where each byte is selected by its own address.

| Operation | Width, capacity, tail, or reset behavior |
|---|---|
| ROM/RAM byte read | Returns the addressed installed byte, otherwise zero. |
| ROM/RAM 16-bit read | At a one-byte tail, returns that byte in bits 0–7 and zero in bits 8–15. Returns zero beyond capacity. |
| ROM/RAM 32-bit read | Assembles remaining bytes into successive low-order byte lanes; unavailable high-order lanes are zero. Returns zero beyond capacity. |
| RAM write | An 8-, 16-, or 32-bit write occurs only when its starting address selects RAM and its entire width fits installed RAM. A partial-width tail write changes nothing. |
| MMIO 8-/16-bit write | Ignored, even when the MMIO attachment is present. |
| MMIO 32-bit write | Delivers the word and the unchanged MMIO-relative byte offset to the attached unit; does nothing when that unit is absent. |
| Logical card reset | Resets the attached MMIO unit, if present. Retains RAM contents and leaves ROM unchanged. |

MMIO reads return zero when no attachment is present. Otherwise, the register read uses the byte offset from `0x30f00000` without rounding that offset to a word boundary. An 8-bit read selects the returned word's byte lane using address bits 1–0. A 16-bit read selects its low or high half using address bit 1. A 32-bit read returns the complete word. These lane rules describe read results; they do not make narrower MMIO writes effective.

Card RAM initially contains zero. A logical card reset retains its contents.

## 1.7 Cartridge mailbox

The cartridge mailbox contains two retained 32-bit words, DATA and CONTROL, and a local interrupt-pending latch. STATUS exposes that latch. IRQ_ACK is a write action rather than a retained register. The mailbox base is `30F00000H` on the selected card. The four word ports are DATA `30F00000H`, CONTROL `30F00004H`, STATUS `30F00008H` and IRQ_ACK `30F0000CH`. Every register reads `00000000H` after mailbox reset. All offsets below are relative to that base.

### 1.7.1 Register map

| Address | Name | Access | Bits / word-write action | Reset read |
|---|---|---|---|---|
| 30F00000H | DATA | R/W | Retains all 32 bits; write replaces the complete word | 00000000H |
| 30F00004H | CONTROL | R/W + trigger | Bits 31:1 retained; bit 0 requests an IRQ and is excluded from readback | 00000000H |
| 30F00008H | STATUS | R; writes ignored | Bit 0 IRQ_PENDING; bits 31:1 zero | 00000000H |
| 30F0000CH | IRQ_ACK | R=0; W action | Any nonzero word clears local pending; zero has no effect | 00000000H |

Read decoding ignores the low two offset bits. Consequently, +0x00 through +0x03 read DATA, +0x04 through +0x07 read CONTROL, and +0x08 through +0x0B read STATUS. Every other offset group reads zero, including +0x0C through +0x0F.

Use aligned word stores for the mailbox actions. Only +0x00, +0x04, and +0x0C perform an action. Writes to STATUS and other aligned offsets are ignored; byte and halfword writes do not perform mailbox actions.

### 1.7.2 DATA and CONTROL

DATA holds the last word written to it. Writing DATA does not change CONTROL or the local interrupt-pending latch.

CONTROL has the following defined fields:

| Bits | Mask | Name | Meaning on a CONTROL write |
|---|---|---|---|
| 0 | 0x00000001 | IRQ_TRIGGER | Requests a local interrupt-pending transition. Not retained by the write. |
| 1 | 0x00000002 | DREQ_READ | Sets the DREQ_READ level. |
| 2 | 0x00000004 | DREQ_WRITE | Sets the DREQ_WRITE level. |
| 31:3 | 0xFFFFFFF8 | Unnamed | Retained exactly as written; no defined IRQ or DREQ contribution. |

A CONTROL write replaces the retained word; it does not set or clear selected retained bits independently. Preserve any retained fields that must remain unchanged when constructing a new CONTROL value. CONTROL readback returns the retained word directly, including unnamed bits. The IRQ_TRIGGER action bit is excluded from the value retained by a CONTROL write, rather than appearing as a latched trigger request in readback.

### 1.7.3 Interrupt pending and acknowledgement

An IRQ_TRIGGER write sets local IRQ_PENDING. A new interrupt edge occurs only when this changes the local latch from clear to set. Repeated trigger writes while IRQ_PENDING is already set do not generate additional edges.

Writing CONTROL with IRQ_TRIGGER clear does not clear IRQ_PENDING. To rearm the local edge, write any nonzero word to IRQ_ACK. A subsequent trigger write can then generate another edge; no intervening trigger-clear CONTROL write is required. IRQ_ACK does not alter DATA or CONTROL. This local acknowledgement is distinct from acknowledgement at a central interrupt controller.

### 1.7.4 DREQ levels and reset

DREQ_READ and DREQ_WRITE independently follow retained CONTROL bits 1 and 2. Either, both, or neither may be asserted. Interrupt triggering and local acknowledgement do not substitute for programming these levels. Unnamed CONTROL bits do not affect them.

Reset clears DATA, CONTROL, and the local pending latch. DATA, CONTROL, and STATUS therefore read zero, and both DREQ levels are deasserted.

## 1.8 Interrupts: two latches, two acknowledgements

Socket 0 raises central interrupt bit 9 (`00000200H`); socket 1 raises bit 10 (`00000400H`). These are distinct from each card's local STATUS bit 0. The central FLAGS port is `08000000H`, ACK is `08000004H` and MASK is `08000008H`. A central ACK clears the pending bits selected by its written ones. It does not clear the card's local pending latch.

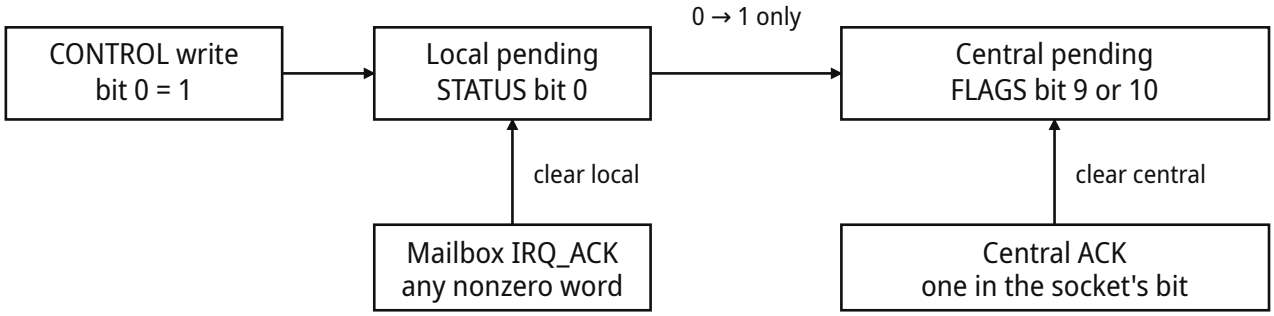



Figure 1.1: A card-local edge and its separate central pending bit

Central pending records an edge even with its MASK bit clear. MASK affects delivery, not pending storage. A FLAGS read changes neither latch. Conversely, mailbox IRQ_ACK clears only the local latch; an already pending central bit remains until central ACK. Neither latch counts repeated events.

For one socket, starting with both pending latches clear:

| Accepted operation | Local pending | Central socket bit |
|---|---|---|
| CONTROL = `1` | 1 | 1 |
| Central ACK = socket mask | 1 | 0 |
| CONTROL = `1` again | 1 | 0 |
| Local IRQ_ACK = `1` | 0 | 0 |
| CONTROL = `1` again | 1 | 1 |
| Local IRQ_ACK = `1` | 0 | 1 |
| Central ACK = socket mask | 0 | 0 |

The third row demonstrates why acknowledging only the central controller does not rearm a mailbox edge. When handling both sockets, select the intended card before touching its local registers and acknowledge only the central bits owned by that handler. The complete CPU-enable, `irq(flags)` and predicate-wait route is in Book I, "Interrupt controller", under "CPU delivery and waiting". For a mailbox protocol, assign DATA and interrupt meanings in the card's own programming specification.

## 1.9 DMA between sockets

A DMA channel can address both cartridge sockets in one transfer without changing the CPU's cartridge selection. Each endpoint's request field supplies its own socket override. `READ_REQUEST` controls the source endpoint; `WRITE_REQUEST` controls the destination endpoint. For either field, selectors 7–10 choose the following request and socket:

| Selector | Request condition | Endpoint socket override |
|---|---|---|
| 7 | Socket 0 `DREQ_WRITE` | Socket 0 |
| 8 | Socket 0 `DREQ_READ` | Socket 0 |
| 9 | Socket 1 `DREQ_WRITE` | Socket 1 |
| 10 | Socket 1 `DREQ_READ` | Socket 1 |

The two overrides are independent of one another and of `CART_SELECT`. Selecting a cartridge request does not constrain the endpoint to a particular cartridge address range: its address may target ROM, card RAM, or mailbox MMIO. The request selects when a block can start and which socket it accesses; the endpoint address selects the region within that socket.

Both programmed request conditions must be asserted before a block is admitted. `FORCE` (selector 0) is always asserted; `DISABLED` (selector 15) never is. Mailbox `CONTROL` bit 1 drives `DREQ_READ`, and bit 2 drives `DREQ_WRITE`. Both may be asserted together. These are request levels, not FIFO occupancy indications or a sequence of per-word requests. An absent card, or a card without a mailbox, supplies no asserted cartridge requests.

For example, a four-word copy from socket 0 to socket 1, incrementing both addresses, uses source selector 8 and destination selector 9:

| `CONTROL` field | Value | Contribution |
|---|---|---|
| Read increment, bit 0 | 1 | `00000001H` |
| Write increment, bit 1 | 1 | `00000002H` |
| Read request, bits 5:2 | 8 | `00000020H` |
| Write request, bits 9:6 | 9 | `00000240H` |
| Block words minus 1, bits 13:10 | 3 | `00000C00H` |
| **Combined `CONTROL`** | | **`00000E63H`** |

Each increment advances four bytes per word. This block requires socket 0 `DREQ_READ` and socket 1 `DREQ_WRITE`; the CPU's current selection cannot reroute either endpoint.

When both requests permit a block to start, DMA captures its addresses, remaining count, control and block length. The block's transfer cost elapses before endpoint accesses occur. At completion, each word is read and then written, after which the channel updates its addresses and count. A zero remaining count completes the channel. Once started, a block runs to completion; a request dropping low holds off the next block rather than shortening the current one.

With selectors other than 7–10, a cartridge endpoint uses `CART_SELECT` at the time of access. Changing SELECT during an active block can therefore redirect that endpoint. Keep selection unchanged until completion, or use an explicit cartridge request selector.

The installed timing basis is 33,868,800 Hz. For a contiguous N-word endpoint span, cartridge service costs `8N + 4` cycles. This includes card RAM and mailbox MMIO: neither receives main-RAM pricing. Main RAM costs `N + 1` cycles. Two cartridge endpoints have additive costs, even when they select different sockets:

| Transfer, wholly within each endpoint's region | Cycles | Four words | Sixteen words |
|---|---|---|---|
| Cartridge ↔ main RAM | `8N + 4` | 36 | 132 |
| Cartridge ↔ cartridge | `16N + 8` | 72 | 264 |
| Cartridge ↔ installed I/O | `8N + 4` | 36 | 132 |

The example ROM(socket 0) → RAM(socket 1) block takes 72 cycles once started. Incrementing across a ROM, RAM or MMIO boundary starts another span and incurs setup again. Add request waiting, channel competition and CPU setup or polling to obtain the complete operation time. Book I, "Direct memory access" gives the general channel programming sequence.

## 1.10 Access faults

CPU alignment checks precede the cartridge bus access. Ordinary halfword accesses require two-byte alignment and word accesses four-byte alignment. A misaligned ordinary load or store enters its address-error exception before reaching the bus. A new mapped fault during an ordinary load enters a data-bus-error exception before committing the read value; a new fault during an ordinary store enters the same exception. A bus-level zero return is not a successful ordinary CPU load of zero. See Book I, “Memory and program storage”, under “Scalar memory accesses”.

Installed I/O and cartridge MMIO have different mapped-access rules. The installed-I/O `CART_SELECT` and `CART_STATUS` registers use word-aligned slots. An unaligned mapped 32-bit access raises `UNALIGNED_IO` (2). An 8-bit read at an aligned register address returns its low byte; other byte lanes fault. All mapped 16-bit reads and all mapped 8-bit or 16-bit stores to installed I/O also fault. Faulting reads return zero and faulting stores are suppressed.

A mapped 32-bit store to `CART_STATUS` raises `READ_ONLY` (3), leaving the register unchanged. This differs from storing to mailbox `STATUS`, which is ignored without that fault.

Cartridge mapped accesses have no aperture-wide word-alignment gate; card decoding and byte lanes determine their effect. Mailbox 8-bit and 16-bit stores are ignored. Mailbox 32-bit stores decode the exact relative offset, so unaligned offsets are ignored rather than rounded to a register.

The entire access must fit inside the cartridge aperture. The highest admitted addresses are `30FFFFFCH` for 32 bits, `30FFFFFEH` for 16 bits, and `30FFFFFFH` for 8 bits. Crossing that final boundary raises `UNMAPPED` (1), returning zero or suppressing the store. An absent selected card returns zero and ignores stores without a fault; ROM stores are likewise ignored.

DMA uses these same mapped-access rules, but a fault does not abort its programmed read, write, and count progression. `DONE` therefore does not prove that endpoint addresses were valid. The first outstanding bus fault retains its code, address, and access; later faults do not replace it. See Book I, “Memory and program storage”, under “Bus-fault status and CPU exceptions” for the fault registers and acknowledgement procedure.

## 1.11 Two-card programming example

Assume a bootable card in socket 0 with ROM and a mailbox, and a data card in socket 1 with at least 32 bytes of RAM and a mailbox. The sequence owns DMA0, both mailbox controls, socket-1 RAM bytes 16–31 and central DMA0 pending bit 0. No other writer changes these resources or the selected request levels. Interrupt delivery remains masked; completion is polled instead of using HALT without an enabled interrupt. No remote processor or automatic mailbox response is assumed.

Declare four words in socket 0’s executable ROM, assert the read request on socket 0 and the write request on socket 1, then program the idle channel:

```
rodata seed: word[4] = {
    0x01234567, 0x89abcdef, 0x55aa33cc, 0xf00dcafe,
}
mem[0x08000008] = 0           -- Mask delivery; retain pending source bits.
mem[0x08000004] = 1           -- Acknowledge this owner's old DMA0 pending bit.
mem[0x08010420] = 0
mem[0x30f00004] = 2           -- Socket 0 DREQ_READ.
mem[0x08010420] = 1
mem[0x30f00004] = 4           -- Socket 1 DREQ_WRITE.
mem[0x0800000c] = seed        -- DMA0 READ_ADDR: socket-0 ROM.
mem[0x08000010] = 0x30000010  -- DMA0 WRITE_ADDR: socket-1 RAM.
mem[0x08000014] = 4           -- Four words, not four bytes.
mem[0x08000018] = 0x00000e63  -- Increment both; requests 8/9; four-word block.
mem[0x08000020] = 1           -- START.
while (mem[0x0800001c] & 1) ~= 0 do
end
```

The explicit endpoint overrides keep the source on socket 0 and destination on socket 1 while ordinary CPU selection remains 1. After normal completion, DMA STATUS is 2 and COUNT is zero. READ_ADDR is `seed + 16`, WRITE_ADDR is `30000020H`, and the masked DMA0 pending bit is set. Before consuming the data, check for new bus faults using the Book I, “Memory and program storage”, under “Attribute a bus fault before START”; DMA DONE does not imply valid endpoint accesses.

Read socket 1’s destination while it remains selected:

```
local destination<const>: *word = 0x30000010
local first = destination[0]
local last = destination[3]
mem[0x30f00004] = 0           -- Deassert socket-1 requests after completion.
mem[0x08010420] = 0
mem[0x30f00004] = 0           -- Deassert socket-0 requests.
mem[0x08000004] = 1
```

The received words are `01234567H`, `89ABCDEFH`, `55AA33CCH`, `F00DCAFEH`; `first` and `last` select the first and fourth. This writes no socket-0 RAM. Logical card reset would retain the destination, so a warm rerun cannot assume that untouched card RAM is zero. The one block takes 72 system cycles, plus CPU setup, request waiting, polling and result reads.